clavity

User Guide · release clavity-v26 (dotnet 0.12.0, classic 0.10.1, agy-autotrain 0.7.1, review-relay 0.3.1, commonmemory 0.4.1) · 2026-10-16

Pair Claude Code with a live Antigravity (`agy`) peer, and keep the pairing sharp.
Each example is tagged **ran** (executed while writing this guide) or **illustrative** (needs a signed-in agy, a network or your Claude Code settings, so it was derived from the docs and source and not run).

Contents

1. Overview: which product do I need?
2. Installation
3. Quick start
4. Core concepts
5. clavity-dotnet in depth
6. clavity-classic in depth
7. The review disciplines (skills)
8. agy-autotrain: teaching the loop
9. commonmemory: a shared notebook
10. review-relay: web-model review rounds
11. Configuration reference
12. Troubleshooting
13. Appendix: command quick reference

1. Overview: which product do I need?

clavity is an umbrella of five independent products. They all revolve around one idea: Claude Code is the **driver** (implements, verifies) and `agy`, Google's Antigravity CLI, is a live **peer** (reviews, recommends). You install only what you need.

Product	What it gives you	Install
clavity-dotnet (primary)	Three MCP tools (`agy_look`, `agy_status`, `agy_ask`) plus the review-discipline skills. Cross-platform, .NET 10.	`claude plugin install clavity@clavity`
clavity-classic (failover)	A Rust CLI, `clavity`, that drives agy through a psmux/tmux "doorbell" and the agentmemory bus. Windows verified; Linux compiles.	Windows installer, or `cargo build --release`
agy-autotrain	Opt-in learning loop: capture what you learn about agy, curate it, inject it into future asks.	`claude plugin install agy-autotrain@clavity`
commonmemory	A convention so Claude and agy tag and recall shared `[common]` notes.	`claude plugin install commonmemory@clavity`
review-relay	Copy-paste review rounds with web AI models (ChatGPT, Gemini, Meta AI, ...).	`claude plugin install review-relay@clavity`

> **dotnet and classic are mutually exclusive.** Pick exactly one driver. The classic installer will not co-install with clavity-dotnet. agy-autotrain, commonmemory and review-relay work alongside either.

The intended workflow (from the project README) is: install superpowers, start with `/brainstorming`, and let the disciplines run. The design document is reviewed by an adversarial panel via agy, then the implementation plan gets the same treatment, then you implement, then `agy-capstone` reviews the committed code and `agy-test-audit` hardens the tests. Any step can be waived by saying so ("Waive agy for this step", or "Waive agy for this and use subagents" if agy is down or out of quota).

The licence is PolyForm Noncommercial 1.0.0: free for personal, academic and non-profit use.

2. Installation

2.1 Prerequisites

You need	For
Claude Code and/or `agy` (Antigravity) installed and signed in	everything
superpowers plugin	auto-firing of the disciplines (without it they stay manually invokable)
.NET 10 SDK	only to build clavity-dotnet from source
`cargo` (Rust), `tmux` / `psmux`	clavity-classic from source
@agentmemory/agentmemory MCP server in *both* Claude and agy	clavity-classic (required), commonmemory (required)
PowerShell 7 (`pwsh`)	review-relay scripts

2.2 Plugins (dotnet, autotrain, commonmemory, review-relay) illustrative

```
claude plugin marketplace add ckir/clavity
claude plugin install clavity@clavity          # clavity-dotnet
claude plugin install agy-autotrain@clavity
claude plugin install commonmemory@clavity
claude plugin install review-relay@clavity
```

clavity-dotnet does not bundle its `clavity-ls` binary. A SessionStart hook fetches the one matching your platform (win-x64, linux-x64, osx-arm64, osx-x64) from the matching `clavity-v<N>` GitHub release the first time a session starts. It is checksum-verified when a `.sha256` asset exists. If the fetch fails (offline, rate-limited) the session start message tells you where to place the binary by hand.

To remove a plugin: `claude plugin uninstall clavity@clavity` (likewise for the others).

2.3 clavity-dotnet from source ran

Run from `clavity-dotnet/` . This is the same single-file publish the release workflow uses, for `linux-x64` :

```
dotnet publish src/Clavity.Cli -c Release -r linux-x64 --self-contained true \
  -p:PublishSingleFile=true -o ./publish
./publish/clavity-ls --version
```

Output

```
clavity-ls 1.0.0+7c69e369
```

(The suffix is the commit the binary was built from.)

2.4 clavity-classic ran

On Windows, download `clavity-classic-setup-<version>.exe` and its `.sha256` from the latest `clavity-v<N>` release (unsigned: SmartScreen will warn, choose More info, then Run anyway). From source, run in `clavity-classic/`:

```
cargo build --release
cp target/release/clavity ~/.local/bin/        # any directory on PATH
clavity --version
```

Output

```
clavity 0.10.1
```

Then do the one-time wiring in section 6: register agentmemory in both agents and give agy its trigger rule.

3. Quick start

3.1 clavity-dotnet: pair Claude with agy in a folder illustrative

```
clavity-ls start ~/projects/myapp
```

This opens a visible agy tab in that folder, waits until agy has paired, then launches Claude Code in the foreground. After 10 seconds without a pairing it reminds you to look at agy's tab (it may be asking you to trust the folder). Ctrl+C aborts. In a folder neither tool has seen before, **both** ask you to trust it, each in its own window; if you do not answer agy's, it never pairs.

Inside Claude Code you do not call anything by hand. Ask in plain language, for example *"get agy's second opinion on this design"*, and Claude uses the `agy_ask` tool.

3.2 clavity-classic: the same, over psmux illustrative

```
clavity C:\projects\myapp          # bare folder = start
clavity -c                         # current folder, forwards -c (continue) to
claude
```

The folder must be the **first** argument. A leading flag means "current folder" and everything is forwarded to `claude`, so write `clavity C:\proj --resume`, not `clavity --resume C:\proj`.

4. Core concepts

- **Driver and peer.** Claude drives, agy peers. A review is advisory: Claude verifies every finding against the real code before folding it in, and verifies the peer's *suggested fix* too.

- **Two transports, one idea.** dotnet talks to agy's local Language Server over gRPC and exposes it as MCP tools. classic types a short "doorbell" into agy's terminal (psmux `send-keys`) and moves the real payload over the agentmemory signal bus.
- `agy_ask` / `clavity ask` **is a write.** It consumes agy quota and posts a visible message in agy's tab. Use look/status/capture to observe for free.
- **Golden header.** Curated agy-driving wisdom that the driver binary prepends to every ask automatically (see section 8). Never prepend it yourself.
- **Disciplines.** Skills that gate decisions and completion: agy-first, adversarial-panel-review, agy-capstone, agy-test-audit.

> **Running economically.** Every turn re-reads the whole context, so a review fired at the end of a long session costs several times what it does fresh (the project measured about 9x the tokens read for identical work). Habits, in order of payoff: (1) implement and commit in one session, then `/compact` or open a fresh chat and say *"run agy-capstone on <range>"*; (2) match the ceremony to the stakes; (3) when you notice a missing test while implementing, just ask for it then. Check `/usage` before a long review.

5. clavity-dotnet in depth

5.1 The MCP tools ran

Claude starts `clavity-ls --mcp` itself (via the plugin's `.mcp.json`); you never run it by hand. To see what it exposes, speak MCP to it over stdio. This drives the real binary (no agy is running here, which is why the status is a failure state):

```
{"jsonrpc":"2.0","id":2,"method":"tools/list"}
```

Output

```
agy_status | Report whether the active agy conversation is idle, working, or
unknown (pre-fire check), plus cascade id and step count. | arguments: none
agy_look   | Look at the active agy conversation's cascade trajectory as a size-
bounded JSON summary (no verbose ids). | arguments: none
agy_ask    | Send a message to the active agy conversation and return agy's reply
(size-bounded JSON) once the conversation goes idle. WRITE: consumes quota and
po... | arguments: message (required), discipline, artifactPath
```

(Output is the tool list reformatted one tool per line, descriptions cut at 150 characters.)

5.2 Reading the results ran

No agy to talk to. `agy_status` called with no paired agy:

Output

{"CascadeId":"","TotalSteps":0,"State":"channel_down","LastStepKind":0,"Diagnostic":{"StatusCode":"LsDiscovery","Detail":"agy Language Server not reachable within 00:00:10 via <scratch>/home/.gemini/antigravity-cli/cli.log; the agy session is still starting or has exited."},"Hint":"clavity-ls -> agy channel call failed ([LsDiscovery] agy Language Server not reachable within 00:00:10 via ..."}

(The `Hint` text is cut here; it continues with advice to check agy's session.)

Fix: start agy with `clavity-ls start <folder>`, or wait for it if it is still starting.

A malformed call is refused before anything reaches agy:

```
{"jsonrpc":"2.0","id":2,"method":"tools/call","params":
{"name":"agy_ask","arguments":{"payload":"hi"}}}
```

Output

```
agy_ask: missing required argument 'message'; unknown argument 'payload'. The arguments are: message (required), artifactPath, discipline. Argument names are case-sensitive.
```

Result	Meaning and what to do
`waiting_for_human`	agy is up but has no conversation yet. Start one in its tab. Do not loop-retry.
`possible_modal`	Idle-wait timed out. agy may be stuck on a modal (auth, quota, consent) *or* the turn is just long. Compare `TotalSteps` before and after: if it moved, wait.
`Answer: null`	Not an error. agy ended its turn on a tool step; read the last assistant step in `Activity`.
`[PEER QUOTA STOP]`	agy ran out of quota during this ask. Wait for the reset it names; retrying early has pushed the reset further out.
`model_unavailable`	The conversation's model is deprecated. Send a message in agy's tab with the model you want.

5.3 Launching on Linux and macOS illustrative

`start` opens agy as a new tab in the terminal you ran it from (mate-terminal, gnome-terminal, konsole, xfce4-terminal), otherwise `xdg-terminal-exec` or `x-terminal-emulator`. Set `CLAVITY_TERMINAL` to choose one. With no display (ssh, say), split it in two:

```
# terminal 1: run agy here
clavity-ls agy ~/projects/myapp
# terminal 2: pair Claude with it (add the session id agy printed if several wait in that folder)
clavity-ls start ~/projects/myapp --attach
clavity-ls start ~/projects/myapp --attach 7f3a9c
```

Each agy accepts one Claude; a second `--attach` to the same agy is refused.

5.4 install, uninstall, is-installed ran

The normal path is `claude plugin install`. `clavity-ls install` is the manual route that registers a plugin with every detected agent. It refuses while Claude Code is running, because a running Claude overwrites the registration. This was run from inside a Claude Code session:

```
clavity-ls install
echo "exit=$?"
clavity-ls is-installed clavity
echo "exit=$?"
```

Output

```
clavity-ls: Claude Code is running — close Claude Code completely, then re-run. A
running Claude overwrites the plugin registration (leaving the bridge
unregistered).
exit=5
clavity is not installed
exit=1
```

`uninstall [--purge-data]` behaves the same way (exit 5 while Claude runs); `--purge-data` also deletes the per-session logs and the golden-header directory.

6. clavity-classic in depth

6.1 One-time wiring illustrative

1. **agentmemory in both agents.** The same MCP server, the same daemon (default `:3111`). For Claude Code: `claude mcp add agentmemory -s user -- npx -y @agentmemory/agentmemory mcp`. Do the equivalent in agy's MCP config. Nothing works without it.
2. **agy's trigger rule.** Put the rule from `clavity-classic/plugin/README.md` ("agy's responder trigger") into `~/.gemini/GEMINI.md`. `clavity start` installs and refreshes the responder skill on every launch, but agy will not fire it without that pointer.
3. **Esc responsiveness (optional).** Add `set -g escape-time 10` and `set -s escape-time 10` to `~/.tmux.conf`; the binary now sets this itself on session creation.

First launch opens a visible watch tab (Windows Terminal) so you can answer agy's login prompts; disable with `AGY_WATCH=0`. Detach with `Ctrl-b d` to hide it; **closing** the tab tears the session down.

6.2 Preflight: doctor and info ran

```
clavity doctor
clavity info
```

Output

```
[ ok ]  tmux/psmux  /usr/bin/tmux
[ ok ]  claude      /home/user/.local/bin/claude
[ ok ]  agy         /home/user/.local/bin/agy
[warn]  session     'claude_agy' = dead (start one with `clavity start <folder>`)
[ -- ]  golden-hdr  none (<scratch>/home/.clavity)

os             = unix
agy_shell      = bash
session        = claude_agy
tmux_bin       = tmux
doorbell       = claudavity: check your inbox and act on any request from claude,
then reply on the bus.
idle_marker    = ? for shortcuts
busy_marker    = esc to cancel
agy_start_args = --dangerously-skip-permissions
```

`dead` only means no session exists yet. Both commands exit 0.

6.3 Observing agy: state, capture, wait-idle ran

These read agy's pane and are free. State is decided by footer text: `? for shortcuts` means idle, `esc to cancel` means busy. To demo without a real agy, a tmux session named `claude_agy` was started with a pane that prints each footer:

```
tmux new-session -d -s claude_agy 'printf "agy> working on it\n\n  ? for
shortcuts\n"; sleep 600'
clavity state
clavity wait-idle --timeout 3
clavity capture --viewport
```

Output

```
idle
idle
agy> working on it

  ? for shortcuts
```

(Assembled from three real runs. `capture --viewport` also printed blank lines down to the bottom of the pane, omitted here.)

With the busy footer instead (`esc to cancel`):

```
clavity state; clavity wait-idle --timeout 2; echo "exit=$?"
clavity cancel
```

Output

```
busy
timeout
exit=1
cancel sent (Escape) to 'claude_agy'
```

Exit code	`wait-idle`
0	idle (prints `idle`)
1	timeout (prints `timeout`)
2	the session is gone

Note that `capture --lines N` counts physical pane lines, trailing blanks included, so on a short pane it may return only blank lines. Use `--viewport` or a larger N.

6.4 Asking agy: req-id, ask, ping ran

You normally do not run these: Claude does, once it knows the protocol (`clavity-classic/docs/agy-remote-control-protocol.md`). They are shown here so you can test the plumbing. A real agentmemory daemon was not available, so a small local stub stood in for the daemon *and* for agy (it answers any request to `agy` with a fixed line). The real `clavity` binary was used unchanged, pointed at the stub with `AGENTMEMORY_URL`.

```
clavity req-id
clavity req-id "refactor foo() to return Result"
```

Output

```
req-dm5wpbpoo8v7
[req_id=req-dm5wpbps1bhj] refactor foo() to return Result
```

```
export AGENTMEMORY_URL=http://127.0.0.1:3199
clavity ask --no-ring "refactor foo() to return Result"
```

Output

```
 INFO sent req-dm5wppo29rwm to agy (signal sig_1, thread Some("thr_1"))
[req_id=req-dm5wppo29rwm] (stub agy) received: 57 chars. No blockers.

[driver_guidance]
Driving the agy peer - TACTICAL reminders only (tendencies; verify against the
live peer). NOTHING HERE IS A SAFETY BOUNDARY, ...
```

The guidance block continues for several more long bullet lines, omitted here (output trimmed to whole lines).

The reply line is on stdout, diagnostics on stderr. The `[driver_guidance]` block is printed on the **first** ask of a session only. The next ask in the same session, with `--review-only`, which prepends the "no edits, verdict only" banner:

```
clavity ask --no-ring --review-only "review src/foo.rs vs the spec; verdict only,
no edits"
```

Output

```
 INFO sent req-dm5wppo7r5fw to agy (signal sig_3, thread Some("thr_3"))
[req_id=req-dm5wppo7r5fw] (stub agy) received: 328 chars. No blockers.
```

The 328 chars against 57 for the same-size instruction shows the banner being added. `ping` is the readiness gate: send `[ping]`, ring, wait for `READY`. Retry it after launch until it exits 0, because agy loads its MCP servers a few seconds after starting:

```
clavity ping --timeout 3
```

Output

```
 INFO sent req-dm5wpyw9txf5 to agy (signal sig_9, thread Some("thr_9"))
 WARN ring debounced: last doorbell 10ms ago < min 2000ms (set
AGY_RING_MIN_INTERVAL_MS=0 to disable)
[req_id=req-dm5wpyw9txf5] READY
```

The debounce warning is a safety feature: rings closer together than 2 seconds are coalesced so a runaway loop cannot burn agy's quota.

6.5 When the plumbing is down ran

```
clavity ask --timeout 2 "hello"            # no daemon listening
clavity await-reply --req-id req-nope --thread-id thr_x --timeout 2
clavity wait-idle --timeout 1              # no session
```

Output

```
agentmemory daemon unreachable at http://127.0.0.1:3111: io: Connection refused
(os error 111)
timeout after 2s waiting for reply to req-nope
session "claude_agy" is gone while waiting for idle
```

Their exit codes, in order, were 2, 1 and 2.

`ask` exits 0 on a reply, 1 on timeout, 2 when the daemon is unreachable. `ping` exits 3 when the ring fails because the session is gone. `await-reply` is for a request you sent yourself over the MCP bus; `--thread-id` is required so the read cannot consume unrelated inbox messages.

6.6 Other sessions, and stopping ran

```
clavity state --session other
clavity stop
clavity stop
```

Output

```
dead
stopped session 'claude_agy'
no session 'claude_agy' to stop
```

> `--session` **goes before or after the subcommand.** `clavity --session other state`, `clavity --session=other state` and `clavity state --session other` all work (checked on the fixed build). Releases up to 0.10.1 misread the first form as "start in a folder named `state`" and failed with `folder not found`; on those, put the flag after the subcommand or set `AGY_SESSION`.

7. The review disciplines (skills)

These ship with the dotnet and classic plugins. Each can be invoked by name in chat, or fired by a hook when superpowers is installed. Each consult is sent review-only: `agy_ask` with `discipline` and `artifactPath` on dotnet, `clavity ask --review-only` on classic. The driver then checks the reply quoted the last line of your artifact, which proves agy read to the end.

Skill	Use it when	Say
`agy-first`	You face a design, scope or sequencing fork. Claude consults agy, verifies its factual claims, then presents **both** agy's recommendation and its own. You decide.	*"run agy-first on this fork"*
`adversarial-panel-review`	A spec or plan was just drafted or materially edited. Multiple expert "seats" each hunt a different defect class over one or more rounds.	*"convene a panel review on docs/spec.md"* (add `--solo` / `--low` for one cheap pass)
`agy-capstone`	Once, before declaring a plan or implementation done. Reviews the **committed code and tests**, round after round, until a round is green.	*"run agy-capstone on main~5..HEAD"*
`agy-test-audit`	After the capstone passes. Asks "would the tests catch the next regression?" Each claimed gap is verified by measurement.	*"run agy-test-audit"*
`open-issues`	Claude noticed a defect that is not the task at hand. It captures one line now; you triage later.	*"start the triage"*
`ls-driving` / `driving`	Reference for how Claude drives agy: look vs ask, banner and phase rules. Loaded automatically.	-

> **Waiving and silencing.** Say "Waive agy for this step" to run one step solo. `.no-agy` in the project root or in `~/.claude/` silences **every** discipline (there is deliberately no per-hook switch); treat it as a last resort. A capstone that stops at its round cap and is waived by you also counts as "gate passed", so `agy-test-audit` asks whether it was a real green.

Hook ownership. A discipline hook shipped by the plugin has exactly one owner: the plugin. If you registered your own hook of the same name in `settings.json`, remove that registration (the file may

stay). Do not register hooks that run from `.clavity/`, the directory agy may write to; the startup check reports any that do.

8. agy-autotrain: teaching the loop

The loop is capture, then curate, then verify, then compile. Capture happens through the `agy-learn` skill the moment you learn something general about agy; curation through `agy-curate`, which drains the inbox (`~/.clavity/agy-observations.md`) into the **GROWTH** region of the golden header and commits it with your driver's `curate-commit` verb. The shipped baseline (**SEED**) is never edited. It needs a driver (dotnet or classic) installed, otherwise nothing injects the header.

8.1 Capture and curate illustrative

```
claude plugin install agy-autotrain@clavity
# in Claude, when you learn something general about agy:
"capture that with agy-learn"
# periodically, when the inbox has grown:
"run agy-curate"
```

8.2 What curate-commit writes ran

The verb reads a compiled region on stdin and atomically writes it plus a SHA-256 sidecar into the golden-header directory (`CLAVITY_GOLDEN_HEADER`, default `~/.clavity`). You do not normally run it; `agy-curate` does after your approval.

```
export CLAVITY_GOLDEN_HEADER=/tmp/gh
printf '## Learned\n- Name the oracle in every delegated task.\n' | clavity
curate-commit
ls /tmp/gh
cat /tmp/gh/golden-header.growth.md.sha256
sha256sum /tmp/gh/golden-header.growth.md
```

Output

```
golden-header.growth.md
golden-header.growth.md.sha256
3a155563fd2d31accb10aea1987b45c1bf9121d4ff32b499a8af9a4af745f39a
3a155563fd2d31accb10aea1987b45c1bf9121d4ff32b499a8af9a4af745f39a  /tmp/gh/golden-
header.growth.md
```

8.3 Integrity check ran

`clavity doctor` reports the header state. After a second commit it reads `active`; after a hand edit it no longer matches its sidecar and is skipped rather than injected:

```
clavity doctor | tail -1
echo tampered >> /tmp/gh/golden-header.growth.md
clavity doctor | tail -2
```

Output

```
[ ok ]  golden-hdr  active (89 content bytes; seed sidecar absent, growth sidecar
present) /tmp/gh
clavity: golden-header region at /tmp/gh/golden-header.growth.md did not match its
.sha256 sidecar — skipped
[ -- ]  golden-hdr  none (/tmp/gh)
```

Never hand-edit the header or its sidecar; re-commit through `curate-commit`. The driver cheatsheet works the same way via `curate-commit --cheatsheet`, writing `driver-cheatsheet.growth.md` and its sidecar. Limits: 32 KiB per region (and combined), 16 KiB for the cheatsheet growth file; over-cap files are skipped with a stderr warning.

8.4 Snapshots and recovery ran

The clavity-ls (dotnet) verb snapshots the previous region before overwriting, keeping the newest five. After two commits:

Output

```
golden-header.growth.md
golden-header.growth.md.42a75fc60dbe31.bak
golden-header.growth.md.42a75fc60dbe31.bak.sha256
golden-header.growth.md.sha256
```

To restore, copy back **both** the file and its sidecar, or the hashes mismatch and the region is silently dropped (PowerShell shown; use `ls -t` and `cp` on Linux):

```
$k = "$HOME\.clavity"
Get-ChildItem "$k\golden-header.growth.md.*.bak" | Sort-Object LastWriteTime -
Descending
Copy-Item "$k\golden-header.growth.md.<stamp>.bak"        "$k\golden-
header.growth.md"
Copy-Item "$k\golden-header.growth.md.<stamp>.bak.sha256" "$k\golden-
header.growth.md.sha256"
```

The observations inbox is restored the same way from `agy-observations.md.<stamp>.bak`; afterwards, delete from `## Pending` anything already present in GROWTH before the next drain. The classic `curate-commit` did not create a snapshot in the same two-commit test.

9. commonmemory: a shared notebook

Both agents already share one agentmemory store; this plugin adds the convention that makes it useful: tag notes `[common]` and recall them at task start. There is no binary and no MCP server of its own.
illustrative (needs a live agentmemory daemon)

```
# Save (handoff, decision, gotcha, fixed bug):
[common] (myapp) — switched CSV export to UTC · Why: reports differed across zones
· Status: done · Next: update docs

# Recall, at the start of a task (the agent runs this tool call):
memory_smart_search query="[common] myapp"
```

Installing the plugin makes the skill *available* but does not make either agent search on its own. Add one line to each agent's global instructions:

- **agy**, in `~/.gemini/GEMINI.md` inside `<user_rules>`: *"At the start of a task, `memory_smart_search` for `[common] <repo>` notes (repo = the current repository name) and read them before acting."*
- **Claude**: the same sentence in your `CLAUDE.md`.

Honour the newest note's `Status:` and ignore stale handoffs. From a clone, install into both CLIs: `claude plugin marketplace add ./commonmemory`, `claude plugin install commonmemory@clavity-commonmemory-dev`, `agy plugin install ./commonmemory`.

10. review-relay: web-model review rounds

For when you want ChatGPT, Gemini, Meta AI or any other site to review a spec, plan or diff. Claude prepares each round; you carry the text and save the replies; Claude collects, verifies each finding against the real text, folds the true ones and starts the next round. Files live in the reviewed project under `.review-relay/<review-name>/`. In normal use you just tell Claude *"start a review-relay review of docs/spec.md"*; the scripts below are what it runs (PowerShell 7). All outputs below were produced by the real scripts in a scratch project.

10.1 Start a spec round ran

```
pwsh -NoProfile -File new-round.ps1 -Review export-spec -Artifact docs/export-
spec.md -NoClipboard
```

Output

```
review-relay: round 1 of 'export-spec' is ready
  upload this file : <scratch>/rrproj/.review-relay/export-spec/round-
01/upload/export-spec.md
  short prompt     : <scratch>/rrproj/.review-relay/export-spec/round-01/prompt-
upload.md
  all-in-one text  : <scratch>/rrproj/.review-relay/export-spec/round-01/prompt-
inline.md
```

Upload the file plus the short prompt to sites that accept files, or paste the all-in-one text where pasting is easier (without `-NoClipboard` it is already on your clipboard). The round's `round.json` records the random end marker the reviewer must quote back:

Output


```
{
  "round": 1,
  "startedAt": "2026-10-16T02:30:57.0338065Z",
  "artifactFile": "export-spec.md",
  "artifactSha256":
"1f222821d6341fb5985ff147b7115ddec78843cbeed2893573ec5bca9c5daa7a",
  "endMarker": "362D-7F12",
  "expectedLastLine": "- A missing `--out` flag prints usage and exits 2.",
  "expectedLineCount": 11
}
```


10.2 Code rounds and guard rails ran

For code, give a git range instead of a file. A range starting with `-` is refused because it would reach git as an option; an uncollected round blocks the next one; names must be lower-case kebab-case:

```
new-round.ps1 -Review export-code -Diff HEAD~1..HEAD -NoClipboard
new-round.ps1 -Review export-bad  -Diff --output=x
new-round.ps1 -Review export-spec -NoClipboard
new-round.ps1 -Review "Bad Name"  -Artifact docs/export-spec.md
```

Output

```
review-relay: round 1 of 'export-code' is ready
  upload this file : <scratch>/rrproj/.review-relay/export-code/round-
01/upload/export-code.diff
  short prompt     : <scratch>/rrproj/.review-relay/export-code/round-01/prompt-
upload.md
  all-in-one text  : <scratch>/rrproj/.review-relay/export-code/round-01/prompt-
inline.md
new-round: refusing git range '--output=x': a range cannot start with '-'
new-round: round 1 has not been collected yet (run collect.ps1, or pass -Force)
Cannot validate argument on parameter 'Review'. The argument "Bad Name" does not
match the "^[a-z0-9][a-z0-9-]*$" pattern. Supply an argument that matches "^[a-z0-
9][a-z0-9-]*$" and try the command again.
```

The last three commands each exit 1.

10.3 Collect ran

`collect.ps1` looks in your AiSave inbox (default Downloads, or `-Inbox` / `REVIEW_RELAY_INBOX`). With nothing saved yet it exits 2:

Output

```
collect: no AiSave captures in <scratch>/inbox saved after 2026-10-
16T02:30:57.0338065Z
```

After a reply was saved (here a hand-written capture in the AiSaveDev format stood in for a real ChatGPT reply, so the verdict below is a test fixture, not a real review), assembled from real runs:

```
pwsh -NoProfile -File collect.ps1 -Review export-spec -Inbox ~/Downloads
```

Output

```
collect: 1 capture(s) -> <scratch>/rrproj/.review-relay/export-spec/round-
01/collected.md

| # | Site | Read proof | Reported lines | Verdict | BLOCKING | MATERIAL | MINOR |
UNKNOWN |
|---|---|---|---|---|---|---|---|---|
| 1 | chatgpt | PASS | - | NOT READY | 1 | 0 | 1 | 0 |
```

(The table is the one in `collected.md`; the rest of that file lays out each reviewer's findings in full.) **Read proof** `PASS` means the reply quoted the end marker and the artifact's last content line. `NO-MARKER` or `MISSING` means a truncated copy or a skipped Step 0. Each finding then gets exactly one disposition in `ledger.md`: `FOLDED`, `REJECTED`, `DUPLICATE`, `DEFERRED` or `OWNER-DECISION`. Stop when every reviewer says `VERDICT: READY`; at round 6 Claude asks whether to continue.

10.4 The capture extension ran

AiSaveDev saves each reply with per-turn markers a reply's own text cannot fake. It is not in the Chrome Web Store, so you load it unpacked once:

```
pwsh -NoProfile -File install-extension.ps1 -Destination ~/relay-ext -WhatIf
pwsh -NoProfile -File install-extension.ps1 -Destination ~/relay-ext
```

Output

```
What if: Performing the operation "install AiSaveDev 0.3.1 (stage it in
<scratch>/ext.new with its marker file, then swap it in)" on target "
<scratch>/ext".
installed AiSaveDev 0.3.1 at <scratch>/ext (first install)
Next, once: open chrome://extensions (or edge://extensions), turn on Developer
mode, click Load unpacked, and pick <scratch>/ext
```

Without `-Destination` it installs to `%LOCALAPPDATA%\review-relay\aisavedev`. After a review-relay update, run it again and click **Reload** on the extension. Plain AiSave also works, but a reply that quotes a `## Human` heading is then misread.

11. Configuration reference

clavity-dotnet / clavity-ls

Variable	Default	Meaning
`CLAVITY_GOLDEN_HEADER`	`~/.clavity`	Directory (not a file) holding the golden-header SEED/GROWTH files.
`CLAVITY_TERMINAL`	auto-detected	Linux: terminal command `start` uses, run by `/bin/sh` with the script path appended, e.g. `gnome-terminal --tab --`.
`CLAVITY_DATA_DIR`	`~/.clavity`	Directory removed by `uninstall --purge-data`; mainly for tests.
`CLAVITY_AGY_IDLE_MAX_SECONDS`	600 (plugin sets 1800)	Ceiling on `agy_ask`'s idle wait. Read only when the MCP server starts.
`CLAVITY_AGY_IDLE_STALL_SECONDS`	120	Seconds with no agy step progress before `possible_modal` (stall). Raise for long single steps.
`CLAVITY_AGY_LOG`, `CLAVITY_SESSION_ID`, `CLAVITY_AGY_ENDPOINT`	set by `start`	Per-session agy log, session id and pairing rendezvous file. Do not set by hand.

clavity-classic / clavity

Variable	Default	Meaning
`AGY_SESSION`	`claude_agy`	psmux session name.
`AGY_TMUX_BIN`	`tmux`	psmux/tmux binary if not on PATH.
`AGY_DOORBELL`	canonical line	Wake string the responder skill keys on.
`AGY_START_ARGS`	`--dangerously-skip-permissions`	Flags `start` passes to agy.
`AGY_WATCH`	enabled	`0` / `false` / `no` disables the first-launch watch tab.
`AGY_IDLE_MARKER` / `AGY_BUSY_MARKER`	`? for shortcuts` / `esc to cancel`	Footer text meaning idle / busy. Change these if an agy update changes its footer.
`AGY_RING_MIN_INTERVAL_MS`	2000	Minimum gap between doorbells; `0` disables.
`AGENTMEMORY_URL`	`http://127.0.0.1:3111`	agentmemory daemon base URL.
`AGENTMEMORY_SECRET`	none	Bearer token, sent only when set.
`RUST_LOG`	`clavity=info`	Log verbosity, e.g. `clavity=debug`.
`CLAVITY_GOLDEN_HEADER`	`~/.clavity`	As above.

review-relay

Setting	Meaning
`REVIEW_RELAY_INBOX`	Where `collect.ps1` looks for captures (default: Downloads).
`.review-relay/templates/spec-review.md` or `code-review.md`	Project-level override of the bundled round prompt.

12. Troubleshooting

Symptom	Cause and fix
`clavity-ls install` / `uninstall` says "Claude Code is running" (exit 5)	Close Claude Code completely, then re-run. Seen in 5.4.
`agy_status` returns `channel_down` / "agy Language Server not reachable"	No paired agy yet, or it exited. Start with `clavity-ls start <folder>`. Seen in 5.2.
agy never pairs in a new folder	agy is waiting for you to trust the folder in its own tab.
"No valid send-model can be resolved"	Send one message in agy's tab with the model you want; changing the dropdown alone does not count.
`golden-header ... did not match its .sha256 sidecar — skipped`	The header was hand-edited or a write was torn. Re-commit with `curate-commit`. Seen in 8.3.
Learned rules quietly stop applying	The driver-cheatsheet growth file is over 16 KiB or lacks its sidecar; republish through `curate-commit --cheatsheet`.
`clavity ask` exits 2, "agentmemory daemon unreachable"	Start the daemon, or set `AGENTMEMORY_URL`. Seen in 6.5.
`clavity --session X state` says "folder not found"	A clavity 0.10.1 or older bug. Update, or put `--session` after the subcommand. See 6.6.
`clavity ping` exits 3	No psmux session to ring. `clavity start <folder>` first.
Hand-started agy: a quota stop looks like a truncated reply	Only agy launched by `clavity start` / `agy` is covered by `[PEER QUOTA STOP]`. Check agy's tab.
Nothing recalls `[common]` notes	agentmemory is not configured in one CLI, or the one-line recall rule is missing from that agent's global instructions.
`collect.ps1` exits 2 / `NO-MARKER` / "no reply"	Wrong inbox (`-Inbox`); a truncated copy (resend the upload variant); a reply quoting `## Human` (re-save with AiSaveDev).
Something agy-facing broke after an agy update	Read `clavity-dotnet/plugin/knowledge/agy-assumptions.md`: each assumption lists how it was verified and how to re-verify. Most fixes are an `AGY_*` env override.

13. Appendix: command quick reference

Command	Does
`clavity-ls start <folder> [--attach [id]] [claude-args]`	agy tab + Claude Code in the folder; `--attach` pairs Claude with an agy from `clavity-ls agy`
`clavity-ls agy [folder]`	Linux/macOS: run agy in this terminal
`clavity-ls --mcp`	MCP stdio server (started by the plugin)
`clavity-ls install [--plugin n]` / `uninstall [--purge-data]` / `is-installed n`	Manual registration (Claude must be closed); exit 0/1 for is-installed
`clavity-ls curate-commit [--cheatsheet]`	Commit a compiled region from stdin
`clavity [folder] [claude flags]` / `start`	Launch agy (psmux) + Claude
`clavity state | capture [--lines N|--viewport] | wait-idle [--timeout N]`	Observe agy: idle/busy/dead, pane text, block until idle
`clavity ring [--no-idle-gate]` / `cancel` / `stop`	Doorbell; send Escape; kill the session
`clavity req-id [text]`	Mint a request id or wrap text in `[req_id=..]`
`clavity ask "text" [--review-only] [--no-ring] [--to A] [--type T] [--timeout N]`	Round trip; exit 0 reply, 1 timeout, 2 daemon down
`clavity await-reply --req-id ID --thread-id T [--timeout N]`	Wait for a reply to a request you sent
`clavity ping [--timeout N]` / `doctor` / `info`	Readiness, preflight, configuration
`new-round.ps1 -Review n (-Artifact path | -Diff range) [-Kind spec|code] [-NoClipboard] [-Force]`	review-relay: start a round
`collect.ps1 -Review n [-Round k] [-Inbox dir]`	review-relay: gather saved replies
`install-extension.ps1 [-Destination dir] [-WhatIf]`	review-relay: stage AiSaveDev

Paths such as `<scratch>` replace a temporary directory in the run outputs, and ANSI colour codes were stripped. Everything marked "ran" was executed against the release clavity-v26 sources on Linux; the Windows-only parts (installer, Windows Terminal watch tab) were not.